

Full Stack Development with MERN

Project Documentation format

1. Introduction :

Project Title:

House Rent Management System.

Team Lead:

S.No	Team Lead Name	Role
1.	Paleti . Smailey	Full Stack Developer

Team Members:

S.No	Team Member Name	Role
1.	Rayala Anjali	Full Stack Developer
2.	Marepalli Kavya	Full Stack Developer
3.	Prattipati Prabhas	Full Stack Developer
4.	Miriyala Harshadu Sri Krishna	Full Stack Developer

2. Project Overview:

. Purpose:

The House Rent Management System is a full-stack web application designed to simplify the process of renting houses by connecting property owners and tenants on a single platform. The project aims to provide an efficient, secure, and user-friendly system for managing property listings, searching rental houses, and handling rental bookings online. It reduces manual work, improves accessibility, and streamlines the overall rental process.

. Features:

- User registration and secure login using JWT authentication.
- Property owners can add, update, and delete house listings.
- Users can browse and search available rental properties.
- Detailed property information with images, price, and location.
- Online house booking/request management.
- Responsive user interface for desktop and mobile devices.
- Secure data storage using MongoDB Atlas.
- Cloud deployment with Vercel (Frontend) and Render (Backend).

3. Architecture:

Frontend Architecture(React.js):

The frontend of the **House Rent Management System** is developed using **React.js**, a modern JavaScript library for building dynamic and responsive web applications. The frontend is responsible for providing an attractive and user-friendly interface through which users interact with the system. It follows a component-based architecture, making the application modular, reusable, and easy to maintain.

React Router is used for client-side navigation, allowing users to move between different pages without reloading the application. React Hooks such as **useState** and **useEffect** are used to manage application state and lifecycle events. The frontend communicates with the backend through RESTful APIs using Axios to send and receive data.

Frontend Components Used:

- Home Page
- Login Page
- Registration Page
- Property Listing Page
- Property Details Page
- Add Property Page
- Edit Property Page
- User Dashboard

- Booking Page
- Search & Filter Component
- Navigation Bar
- Footer

Where the Frontend is Used:

The frontend provides the interface for all user interactions, including:

- User Registration and Login
- Viewing available rental properties
- Searching and filtering properties
- Viewing complete property details
- Adding new property listings
- Editing and deleting property information
- Booking rental properties
- Managing user profile and dashboard
- Displaying booking history and rental information

The frontend is deployed on **Vercel**, enabling fast loading, automatic deployment, and high availability.

Backend Architecture(Node.js and Express.js):

The backend is developed using **Node.js** and **Express.js**. It serves as the core processing unit of the application by handling business logic, processing client requests, authenticating users, validating data, and communicating with the MongoDB database.

The backend follows the REST API architecture, where each API endpoint performs a specific operation. Express.js handles routing, middleware execution, request validation, and response generation. Secure authentication is implemented using **JWT (JSON Web Token)**, while user passwords are encrypted using **bcrypt.js**.

Backend Modules Used:

- Authentication Module
- User Management Module
- Property Management Module
- Booking Management Module
- API Routing Module

- Database Connectivity Module
- JWT Authentication Module

Where the Backend is Used:

The backend performs all server-side operations, including:

- User Registration and Login Authentication
- JWT Token Generation and Verification
- Password Encryption using bcrypt.
- Adding, Updating, and Deleting Property Listings
- Processing Booking Requests
- Validating User Input
- Fetching Property Data
- Handling CRUD Operations
- Sending JSON Responses to the Frontend
- Connecting to MongoDB Atlas

The backend is deployed on **Render Cloud**, ensuring secure and scalable API services.

Database Architecture (MongoDB):

The application uses **MongoDB Atlas**, a cloud-based NoSQL database that stores all application data in document format. MongoDB provides high performance, scalability, and flexibility for handling dynamic data.

The backend communicates with MongoDB using **Mongoose**, which provides schema validation, object modeling, and simplified database operations.

Database Operations:

The database supports all CRUD operations:

- Create new user accounts
- Store property listings
- Update property details
- Delete unavailable properties
- Retrieve property information
- Store booking requests
- Update booking status
- Manage user profile information

Where the Database is Used:

The MongoDB database is used to:

- Store registered user information
- Store property details and uploaded images
- Store booking requests and booking history
- Retrieve property information for users
- Update user profiles and property records
- Maintain secure and consistent application data

MongoDB Atlas provides cloud-based storage, automatic backups, secure authentication, and scalable data management for the application.

4. Setup Instructions :

. Prerequisites:

Before setting up the **House Rent Management System**, ensure that the following software and tools are installed on your system:

- **Node.js**: Required to run the backend server and manage project dependencies.
- **npm (Node Package Manager)**: Used for installing and managing required packages.
- **MongoDB**: Used as the database for storing user details, house rental information, bookings, and other application data.
- **Git**: Required to clone the project repository from GitHub.
- **Code Editor**: Visual Studio Code or any preferred IDE for modifying and running the project.
- **Web Browser**: Google Chrome, Microsoft Edge, or any modern browser to access the frontend application.
- **Postman (Optional)**: Used for testing backend REST APIs.
- **Internet Connection**: Required for installing dependencies and accessing deployed services.

.Installation Steps:

Step 1: Clone the Repository .

Clone the project from GitHub using the following command.

```
git clone  
https://github.com/your-username/house-rent-management.git
```

Move into the project directory.

```
cd house-rent-management
```

Step 2: Install Frontend Dependencies:

Navigate to Frontend Folder:

```
cd frontend
```

Install required React packages:

```
npm install
```

Step 3: Install BackEnd Dependencies:

Navigate to Backend Folder:

```
cd backend
```

Install required Node.js packages:

```
npm install
```

Step 4: Configure Environment Variables

Create a **.env** file inside the backend folder and add the required configuration values:

example:

```
PORT=5000
```

```
MONGO_URI=<your_mongodb_connection_string>
```

JWT_SECRET=<your_secret_key>

Step 5: Connect Database

The House Rent Management System uses MongoDB as its database to store and manage application data such as user details and rental information.

The MongoDB connection is established by adding the MongoDB Atlas connection URL in the backend `.env` file using the `MONGO_URI` variable.

Once the backend server is started, the application connects with MongoDB and enables data storage and retrieval operations.

Step 6: Start the Application:

Run Backend Server

`npm start`

Run Frontend Server

`npm start`

After successfully starting both servers, the React frontend communicates with the Node.js backend, which connects to MongoDB to store and retrieve house rental management data. The complete application will be ready for users to register, login, and manage rental information

5. Folder Structure:

The **House Rent Management System** follows a structured full-stack project organization, separating the frontend and backend components. This improves maintainability, scalability, and efficient development.

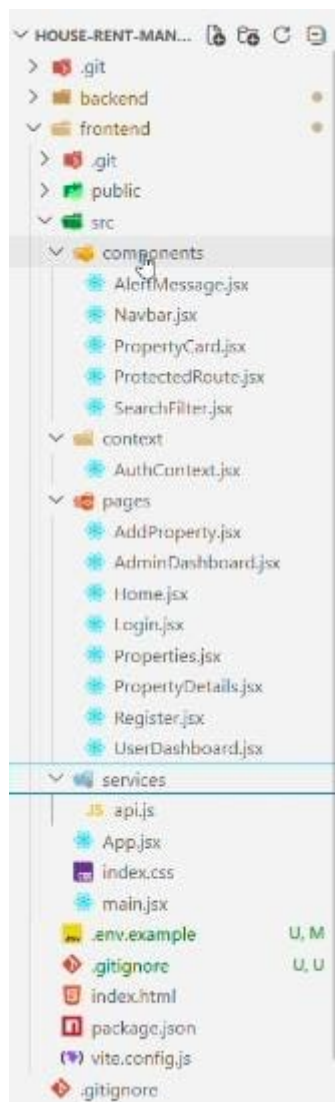
Client (React Frontend) Folder Structure

The **Client** folder contains the React.js application responsible for designing the user interface and handling user interactions

Client Folder Description:

- **components/**: Contains reusable React components such as navigation bars, buttons, forms, and common UI elements.
- **pages/**: Includes different application screens such as user registration, login, home page, dashboard, and rental house pages.
- **services/**: Handles API requests and communication between the React frontend and backend server.
- **assets/**: Stores images, icons, and other static files used in the application.
- **App.js**: Main React component that manages application routing and structure.
- **package.json**: Contains frontend dependencies and project configuration.
- **.env**: Stores frontend environment variables such as backend API URL.

Client Folder Structure:



Server (Node.js Backend) Folder Structure

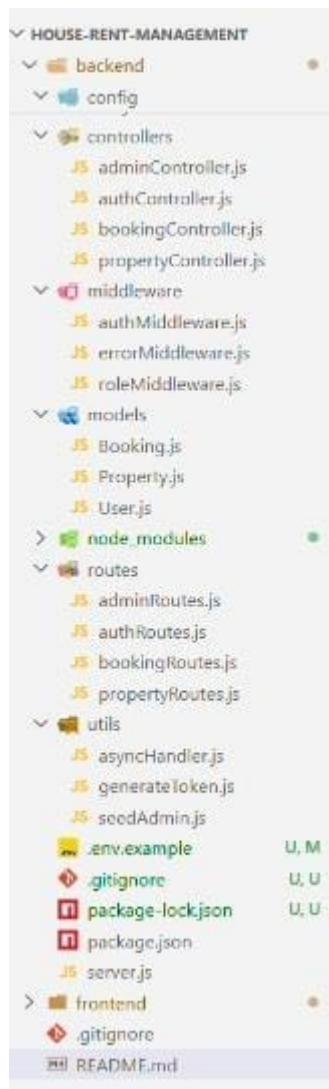
The **Server** folder contains the Node.js and Express.js backend application that manages APIs, authentication, database operations, and business logic.

Server Folder Description:

- **config/**: Contains database configuration files used to establish MongoDB connectivity.
- **controllers/**: Handles application logic such as user authentication, house management, and rental operations.
- **models/**: Defines MongoDB schemas for storing user details, house information, and rental data.

- **routes/**: Defines API endpoints for communication between frontend and backend.
- **middleware/**: Contains authentication and security-related functions.
- **server.js**: The main entry point that initializes the Express server and connects all backend components.
- **package.json**: Contains backend dependencies and scripts required to run the server.
- **.env**: Stores sensitive configuration details such as database URL, port number, and secret keys.

Server Folder Structure:



6. Running the Application:

To run the **House Rent Management System** locally, both the frontend and

backend servers must be started separately. The frontend handles the user interface, while the backend provides APIs and communicates with the MongoDB database.

Frontend Server

Navigate to the **client** directory:

```
cd client
```

Install the required dependencies:

```
npm install
```

Start the React frontend application:

```
npm start
```

The frontend application will run locally at:

```
http://localhost:3000
```

Backend Server

Open another terminal and navigate to the **server** directory:

```
cd server
```

Install the required backend dependencies:

```
npm install
```

Start the Node.js backend server:

```
npm start
```

The backend server will run locally at:

```
http://localhost:5000
```

After successfully running both servers, the React frontend communicates with the Node.js/Express backend through API requests. The backend processes the requests, interacts with the MongoDB database, and returns the required data to the frontend application. The complete **House Rent Management System** will be available for users to register, login, view

rental properties, and manage house rental information.

7. API Documentation:

The **House Rent Management System** backend provides RESTful APIs developed using **Node.js and Express.js**. These APIs handle user authentication, house management, and rental-related operations. The frontend communicates with these APIs to send and receive data from the MongoDB database.

Base URL:

<https://house-rent-management-backend-bkow.onrender.com>

User Registration

Endpoint:

POST /api/auth/register

Description:

Creates a new user account in the system.

Request Body:

```
{  
  "name": "John Doe",
```

```
"email": "john@gmail.com",  
"password": "password123"  
}
```

Response:

```
{  
  "message": "User registered successfully",  
  "user": {  
    "name": "John Doe",  
    "email": "john@gmail.com"  
  }  
}
```

User Login**Endpoint:**

POST /api/auth/login

Description:

Authenticates the user and generates an access token.

Request Body:

```
{  
  "email": "john@gmail.com",  
  "password": "password123"  
}
```

Response:

```
{  
  "message": "Login successful",  
  "token": "jwt_token",  
  "user": {  
    "email": "john@gmail.com"  
  }  
}
```

Add New House

Endpoint:

POST /api/houses

Description:

Allows users to add rental house details.

Request Body:

```
{  
  "title": "2BHK House",  
  "location": "Hyderabad",  
  "rent": 15000,  
  "description": "Spacious house with parking facility"  
}
```

Response:

```
{  
  "message": "House added successfully",  
  "house": {  
    "title": "2BHK House",  
    "location": "Hyderabad",  
    "rent": 15000  
  }  
}
```

Get All Houses

Endpoint:

GET /api/houses

Description:

Retrieves all available rental houses from the database.

Response:

```
[  
  {  
    "title": "2BHK House",  
    "location": "Hyderabad",  
    "rent": 15000  
  },  
  {  
    "title": "1BHK Apartment",  
    "location": "Vijayawada",  
    "rent": 9000  
  }  
]
```

Get House By ID

Endpoint:

GET /api/houses/:id

Description:

Fetches details of a specific house using its ID.

Response:

```
{  
  "_id": "65abc123",  
  "title": "2BHK House",  
  "location": "Hyderabad",  
  "rent": 15000  
}
```

Update House Details:

Endpoint:

PUT /api/houses/:id

Description:

Updates existing house rental information.

Request Body:

```
{
```

```
"rent": 17000,  
"location": "Secunderabad"  
}
```

Response:

```
{  
  "message": "House details updated successfully"  
}
```

Delete House:**Endpoint:**

DELETE /api/houses/:id

Description:

Deletes a house listing from the system.

Parameters:

id : House unique identifier

Response:

```
{
```

```
"message": "House deleted successfully"
}
```

Create Rental Request

Endpoint:

POST /api/rentals

Description:

Allows users to send rental requests for available houses.

Request Body:

```
{
  "houseId": "65abc123",
  "userId": "65xyz456",
  "message": "I am interested in this house"
}
```

Response:

```
{
  "message": "Rental request submitted successfully"
}
```

Get Rental Requests

Endpoint:

GET /api/rentals

Description:

Retrieves rental requests from the database.

Response:

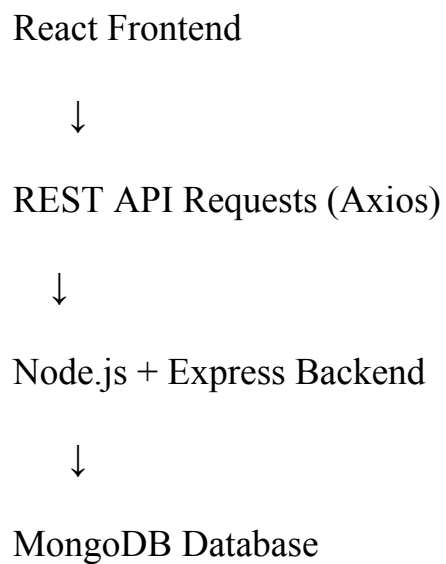
```
[  
  {  
    "houseId": "65abc123",  
    "status": "Pending"  
  }  
]
```

API Error Response

if an invalid request is sent, the API returns an error response:

```
{  
  "success": false,  
  "message": "Invalid request or data not found"  
}
```

API Communication Flow



The API layer acts as a bridge between the frontend and database, allowing users to securely register, authenticate, manage rental properties, and perform rental operations in the **House Rent Management System**.

8. Authentication:

The **House Rent Management System** implements authentication and authorization using **JSON Web Token (JWT)** to provide secure access to the

application. Authentication ensures that only registered users can access the system, while authorization controls access to protected resources based on user permissions.

User Authentication Process

The authentication process works as follows:

1. Users create an account by providing required details such as name, email, and password through the registration API.
2. User passwords are encrypted using **bcrypt** before storing them in the MongoDB database.
3. During login, the entered credentials are verified with the stored user data.
4. If the credentials are valid, the backend generates a **JWT token** and sends it to the frontend.
5. The frontend stores the token and uses it for accessing protected routes and API requests.

JSON Web Token (JWT) Authentication:

The project uses **JWT-based authentication** for maintaining secure communication between the React frontend and Node.js backend.

- JWT tokens are generated after successful user login.
- The token contains user identification information and is signed using a secret key stored in the backend `.env` file.
- The token is sent with every protected API request through the authorization header.
- The backend verifies the token before allowing access to restricted operations.

Example Authorization Header:

Authorization: Bearer <JWT_Token>

Password Security:

- User passwords are not stored as plain text in the database.
- **bcrypt** is used to hash passwords before saving them in MongoDB.
- During login, bcrypt compares the entered password with the encrypted password stored in the database.

- This ensures secure storage and protection of user credentials.

Authorization Handling:

Authorization is implemented using middleware in the backend.

- Protected routes check whether a valid JWT token is provided.
- Only authenticated users can perform operations such as:
 - Adding house listings
 - Updating house details
 - Deleting rental properties
 - Managing rental requests
- Unauthorized users are restricted from accessing protected resources.

Authentication Flow:

User Login



React Frontend Sends Credentials



Node.js + Express Backend Validates User



JWT Token Generated



Token Sent to Frontend



Token Included in Future API Requests



Backend Verifies Token



Access Granted to Protected Resources

The JWT-based authentication system provides secure user management and ensures that only authorized users can access and manage features of the **House Rent Management System**.

9. User Interface:

Fig 1: Home page/main page.

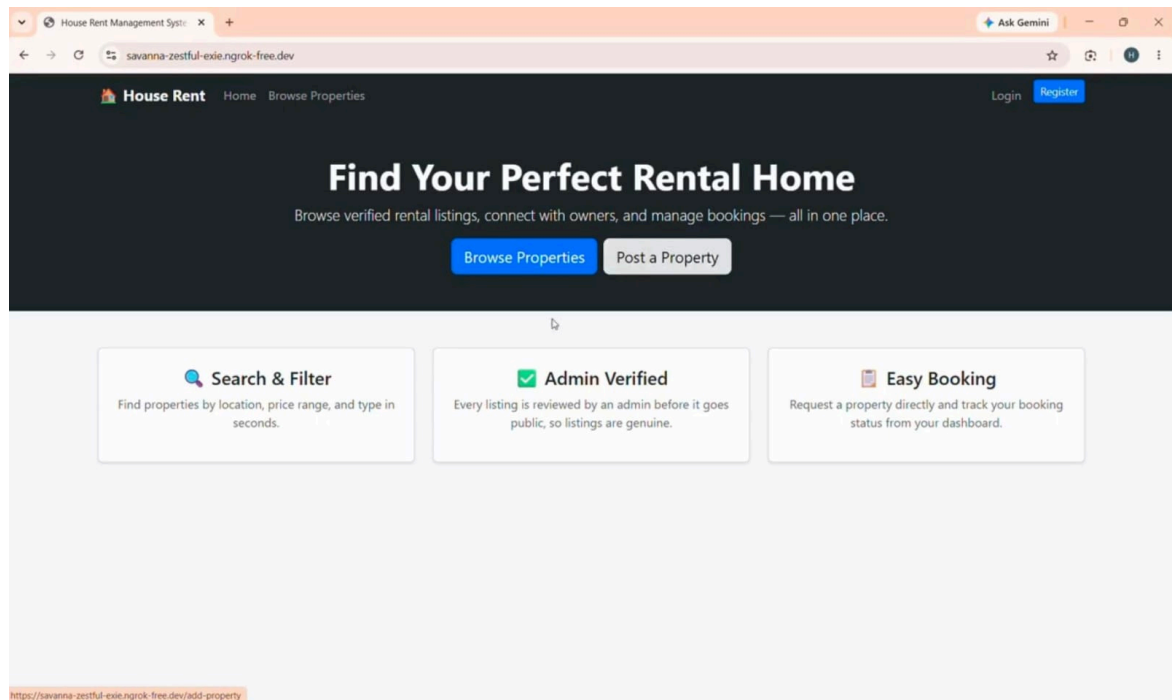


Fig 2:creating an account(register).

The screenshot shows a web browser window with the URL `savanna-zestful-exie.ngrok-free.dev/register`. The page title is "House Rent Management System". The navigation bar includes "Home" and "Browse Properties" links, and "Login" and "Register" buttons. The main content area features a "Create an Account" form with the following fields: "Full Name" (containing "Harshadu"), "Email" (containing "srikrishnamiriyala99@gmail.com"), "Phone (optional)" (containing "9876843215"), and "Password" (containing "*****"). A blue button labeled "Creating account..." is positioned below the password field. Below the button, there is a link that says "Already have an account? [Login](#)".

Fig 3:making changes in profile and saving the changes.

The screenshot shows a web browser window with the URL `savanna-zestful-exie.ngrok-free.dev/dashboard`. The page title is "House Rent". The navigation bar includes "Home", "Browse Properties", and "Post Property" links, and "Hi, Harshadu" and "Logout" buttons. The left sidebar contains a "My Dashboard" section with links for "My Profile", "My Listings", and "My Bookings". The main content area features a "My Profile" form with the following fields: "Email" (containing "srikrishnamiriyala99@gmail.com (cannot be changed)"), "Name" (containing "Harshadu"), "Phone" (containing "9876843215"), and "New Password (leave blank to keep current)". A blue button labeled "Save Changes" is positioned below the password field.

Fig 4:giving the details of the house rent.

House Rent Management System

savanna-zestful-exie.ngrok-free.dev/add-property

House Rent Home Browse Properties Post Property Hi, Harshadu Logout

Description

A house with 4 rooms.

Property Type

House

Monthly Rent (₹)

8000

Bedrooms

2

Bathrooms

2

Area (sqft)

77

Location

City

mylavaram

State

Andhra Pradesh

Address

LBRCE college,mylavaram

Pincode

522222

Image URLs (comma-separated, optional)

https://example.com/img1.jpg, https://example.com/img2.jpg

File upload is a planned future enhancement — for now, paste image links.

Submit for Approval

Fig 5:checking the progress in my listing.

House Rent Management System

savanna-zestful-exie.ngrok-free.dev/dashboard

House Rent Home Browse Properties Post Property Hi, Harshadu Logout

My Dashboard

- My Profile
- My Listings
- My Bookings

My Listings

+ Post New Property

Title	City	Rent	Status	Actions
Individual House	mylavaram	₹8000	Pending	View Delete

Fig 6:showing the details of a rent house .

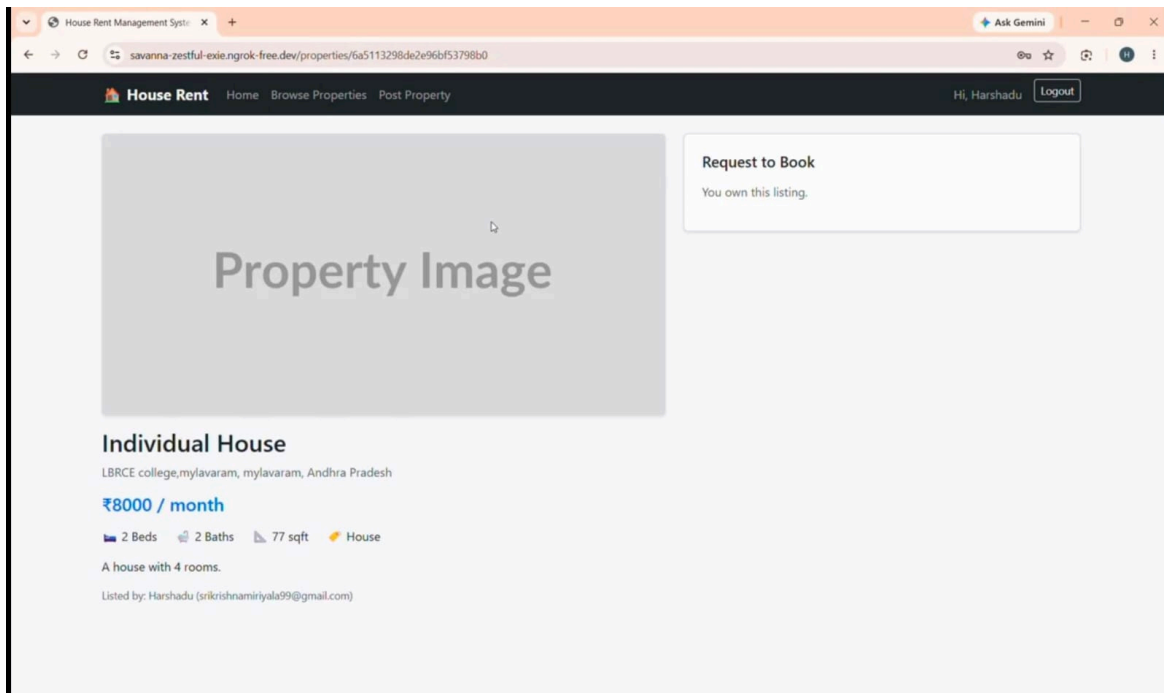


Fig 7:login credentials.

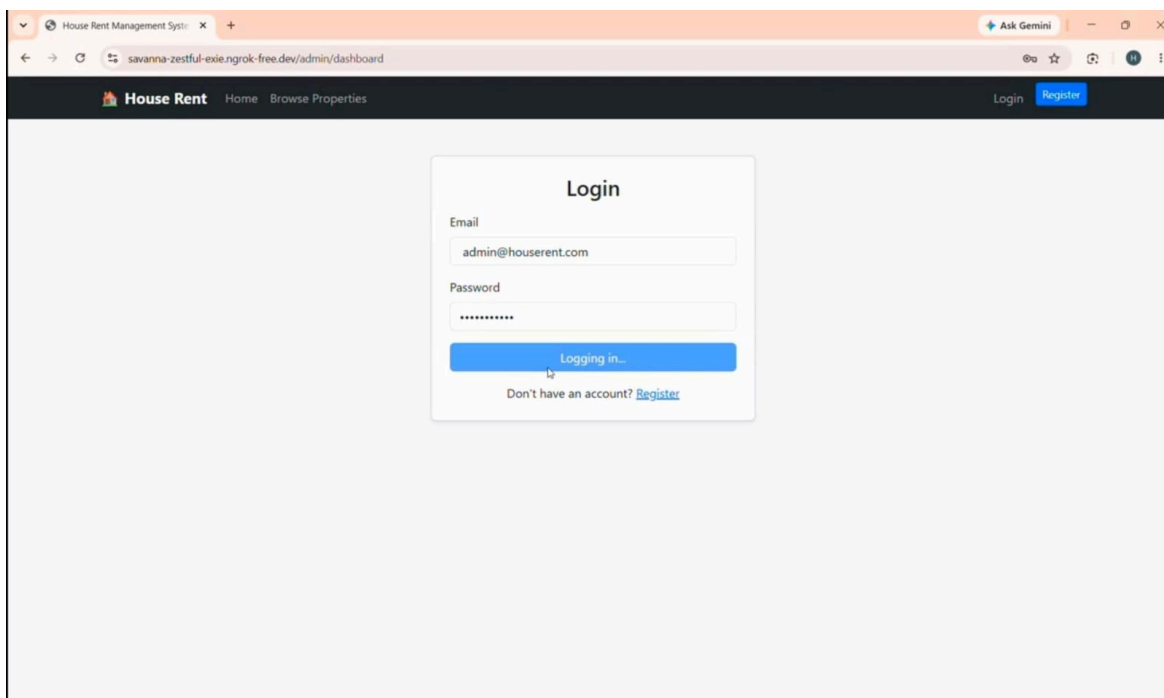


Fig 8:showing the status the admin panel.

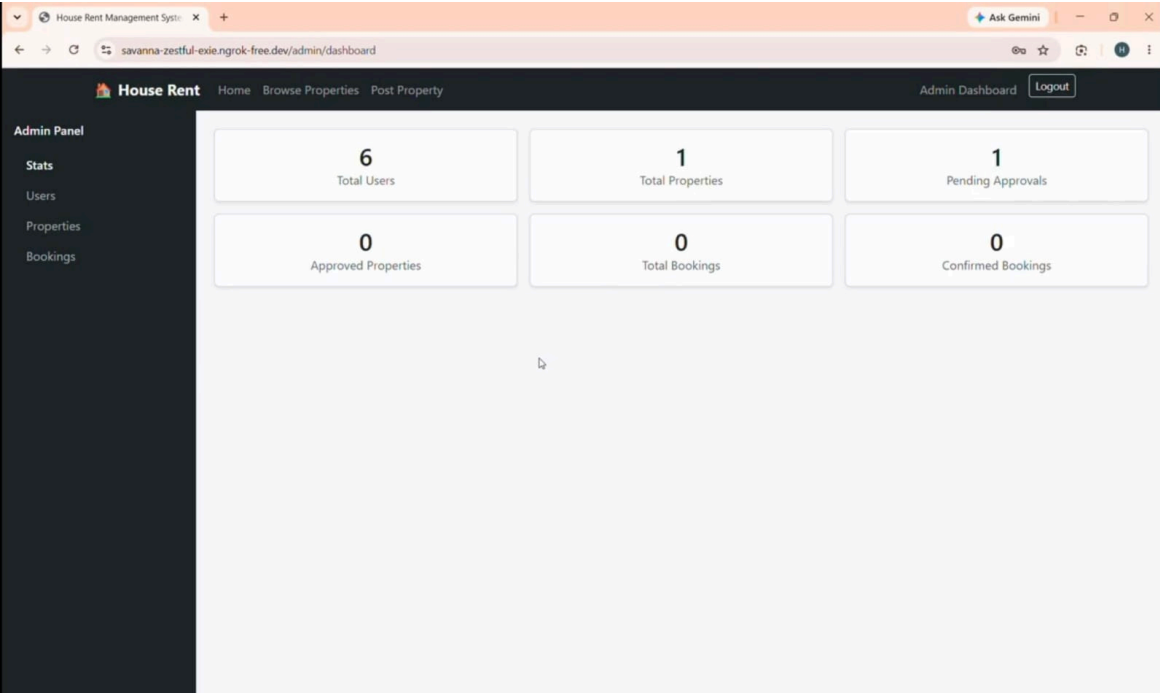


Fig 9:managing the users.

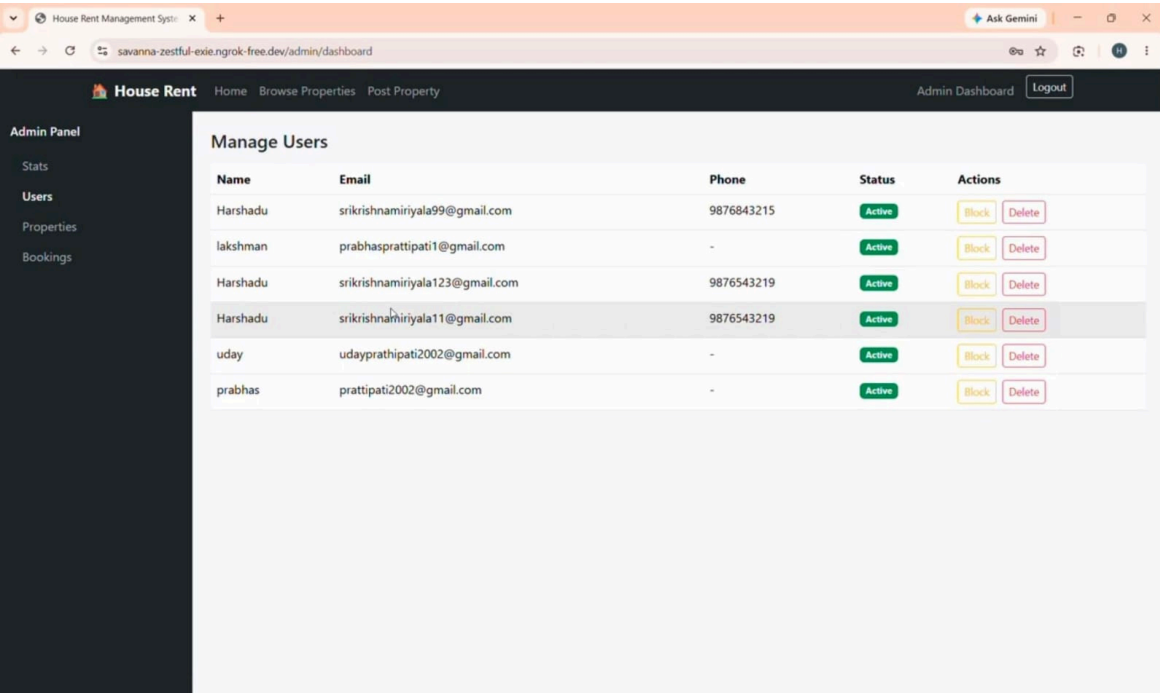


Fig 10:managing the properties.

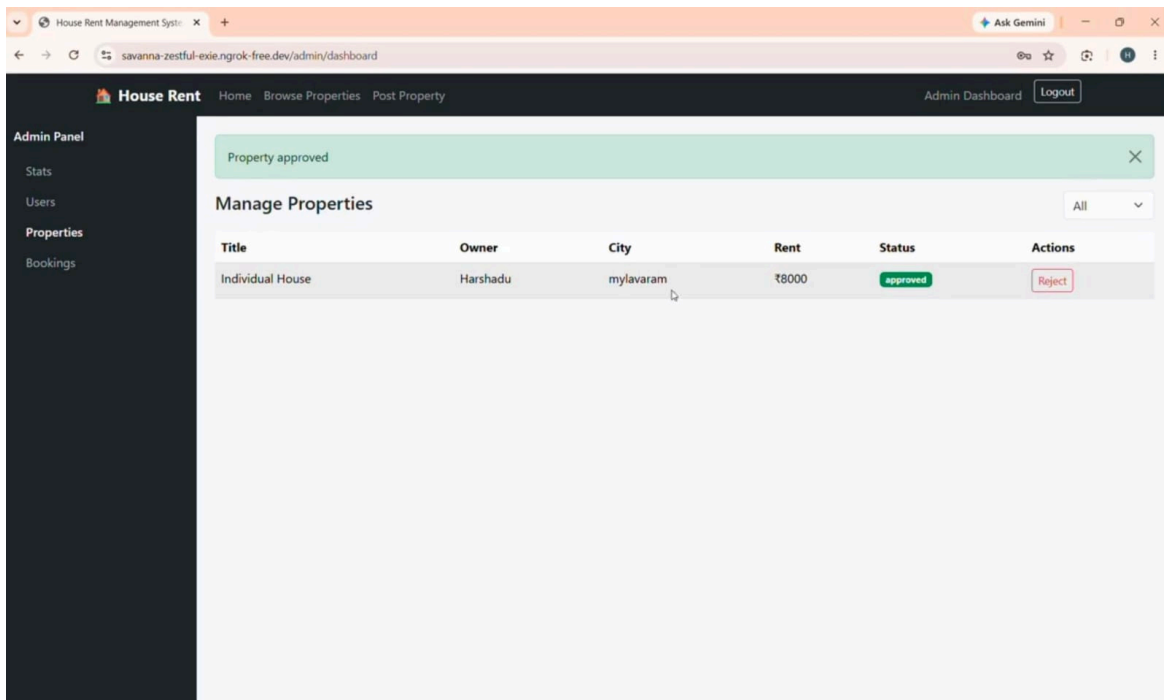


Fig 11: Showing the browse rental properties.

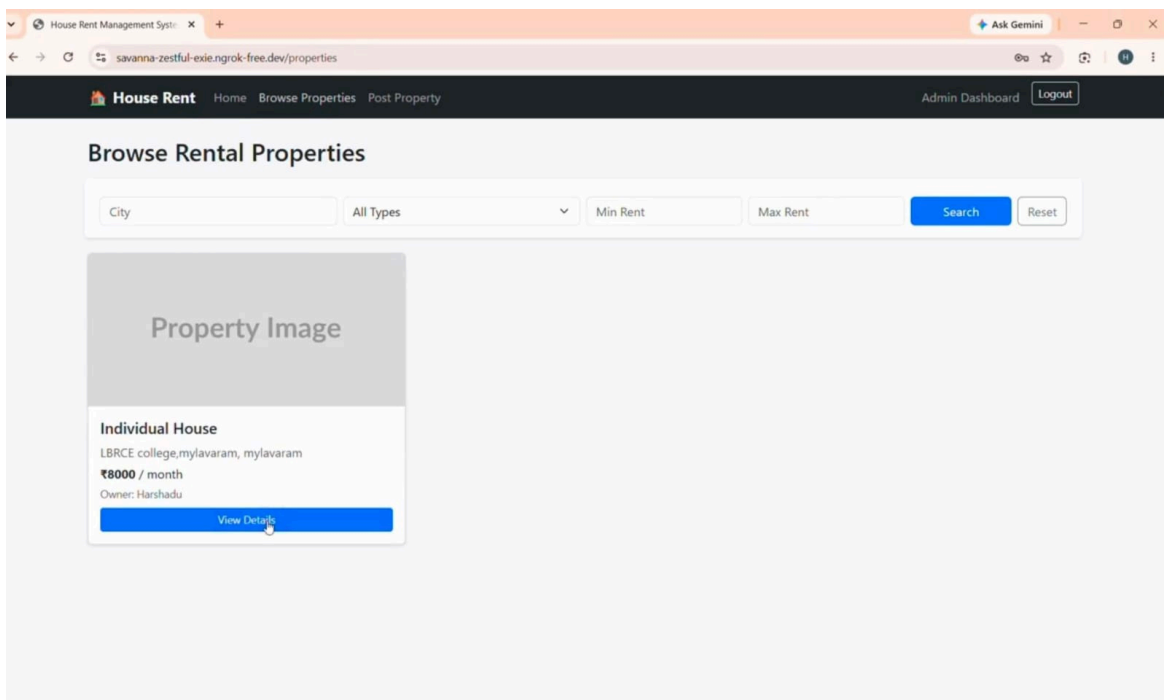
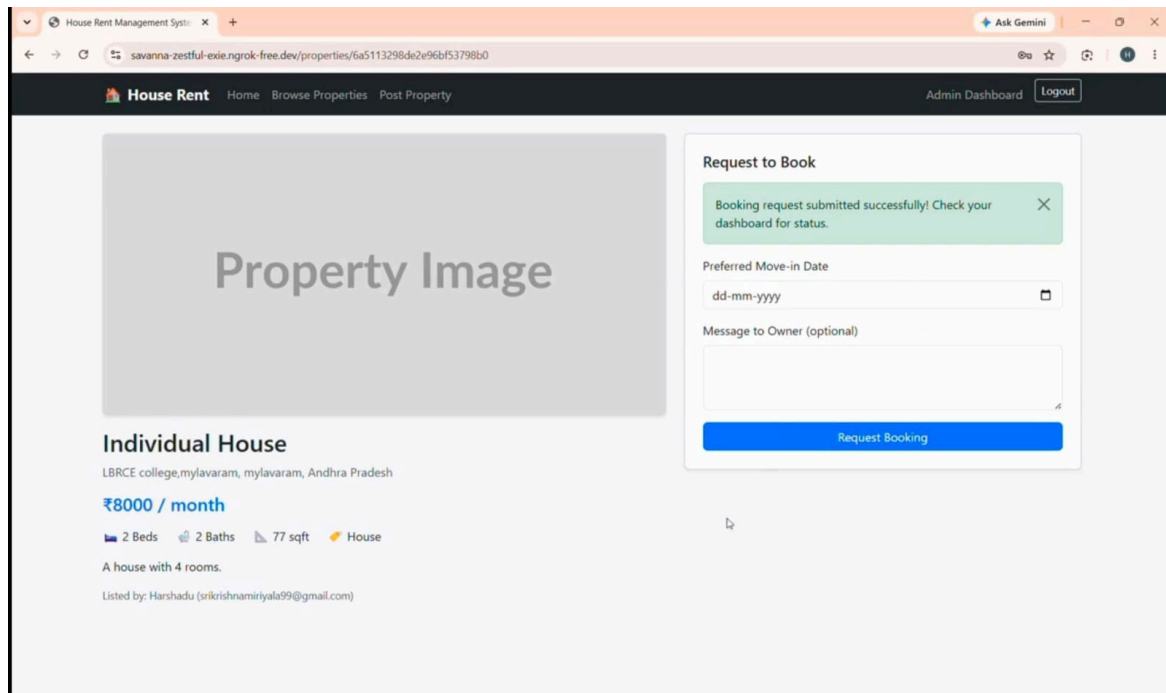


Fig 12: here we need to request book the house for rent.



10. Testing:

The **House Rent Management System** follows a systematic testing approach to ensure the application works correctly, securely, and efficiently.

- **Functional Testing** was performed to verify the major features of the application, including user registration, login, house listing, updating house details, deleting properties, and rental request management.
- **Frontend Testing** was carried out to check the React user interface, page navigation, form validations, and user interactions.
- **Backend Testing** was performed to verify REST API endpoints, request handling, authentication, and database operations.
- **API Testing** was conducted using **Postman** to test backend APIs by sending different HTTP requests such as GET, POST, PUT, and DELETE.
- **Database Testing** was performed to ensure that user details, house information, and rental data are stored and retrieved correctly from MongoDB.
- **Authentication Testing** was done to verify JWT token generation, validation, and access control for protected routes.
- **Error Handling Testing** was performed to ensure the system displays appropriate messages for invalid inputs, unauthorized access, and failed requests.
- **Deployment Testing** was conducted after deploying the application on **Vercel (Frontend)** and **Render (Backend)** to verify the live

application functionality.

Testing Tools Used:

- **Postman** – Used for testing and validating backend REST APIs.
- **MongoDB Compass / MongoDB Atlas** – Used for checking database records and operations.
- **Web Browser (Google Chrome)** – Used for testing frontend functionality and user experience.
- **VS Code** – Used for debugging and resolving application errors during development.

11. Screenshots or Demo :

The House Rent Management System is a full-stack web application developed using the MERN stack (MongoDB, Express.js, React.js, and Node.js). The application allows users to browse rental properties, view detailed house information, register and log in securely, and manage rental listings through an intuitive and responsive interface.

Live Demo

<https://house-rent-management-frontend-ten.vercel.app/>

The deployed application demonstrates the complete workflow of the House Rent Management System, including user authentication, property management, browsing rental listings, and secure communication between the React frontend and the Node.js/Express backend using MongoDB as the database.

12. Known Issues:

1. Server Startup Delay

- The backend server hosted on Render may take some time to respond after a period of inactivity due to server sleep behavior.

2. Internet Dependency

- The application requires a stable internet connection to communicate with the deployed backend server and MongoDB database.

3. Image Loading Performance

- Property images may take additional time to load depending on image size and network speed.

4. Limited Search and Filtering

- The current version provides basic property search functionality, while advanced filtering options can be added in future updates.

5. JWT Token Expiration

- Users may need to log in again when the authentication token expires.

6. Data Validation

- Incorrect or incomplete user inputs during registration or property listing may cause validation errors.

7. No Real-Time Notifications

- The system currently does not support real-time notifications for rental requests, approvals, or updates.

8. Third-Party Service Dependency

- The application depends on external services such as Vercel, Render, and MongoDB Atlas, and temporary service issues may affect availability.

9. Responsive Design Improvements

- Some UI elements may require further optimization for better display on different screen sizes.

10. Future Feature Enhancements

- Additional features such as online rent payment, property location maps, tenant-owner chat, and advanced analytics can be implemented in future versions.

13. Future Enhancements:

1. Online Rent Payment Integration

- Integrate secure online payment gateways to allow tenants to pay rent directly through the application.

2. Advanced Property Search and Filtering

- Add advanced search options such as location, rent range, property type, number of rooms, and available facilities to help users find suitable houses easily.

3. Real-Time Notifications

- Implement real-time notifications for rental requests, approvals, new property listings, and important updates.

4. Map Integration

- Add map-based location services using APIs to display exact property locations and nearby facilities.

5. Chat System Between Users

- Develop a real-time messaging feature that allows tenants and property owners to communicate directly within the application.

6. Property Reviews and Ratings

- Allow tenants to provide reviews and ratings for properties and owners to improve trust and decision-making.

7. Mobile Application Development

- Develop Android and iOS mobile applications to provide easier access and better user experience.

8. AI-Based Property Recommendations

- Implement AI-based recommendation systems to suggest houses based on user preferences, budget, and previous searches.

9. Admin Dashboard

- Create an admin panel to monitor users, manage property listings, handle complaints, and analyze system activities.

10. Enhanced Security Features

- Add additional security features such as two-factor authentication, improved data encryption, and enhanced user privacy protection.